PROJECT REPORT

IMPLEMENT CLIENT SCRIPT & UI POLICY (INCIDENT)

Date	3 October 2026
Team ID	SWTID-2026-3849
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Mark	

1. INTRODUCTION

1.1 Project Overview

This project configures ServiceNow Incident Management using a UI Policy, a UI Policy Action, and Client Scripts. These configurations guide users as they create or update incident records, control selected field behaviour, validate information before saving, and prevent a specified State update through inline list editing.

The project runs within a ServiceNow Personal Developer Instance (PDI). Users interact with the standard Incident form and Incident list through a web browser. The implementation is a ServiceNow configuration project; the supplied project material does not describe a separate React frontend, Express backend, or MongoDB database.

1.2 Purpose

The purpose is to improve the consistency of incident data entry and provide immediate feedback to service-desk agents. The configuration responds to changes in Impact, applies the documented Urgency and Priority behaviour, checks the Assigned to requirement for High-impact incidents, and blocks the specified inline State edit.

This report documents the problem, ideation, requirements, design, planning, functional testing, results, advantages and limitations, future scope, and appendix information using the provided Project Report Format.

2. IDEATION PHASE

2.1 Problem Statement

Incident records may contain missing or inconsistent information. In particular, a High-impact incident may require additional assignment information, and combinations of Impact, Urgency, and Priority should follow the configured handling rules. Users may also edit a field directly from the Incident list, where mistakes can be made quickly.

These issues can lead to inconsistent records, additional correction work, and a less predictable user experience. The project addresses them with client-side field behaviour, validation messages, and a restriction on the documented State list edit.

2.2 Empathy Map Canvas

THINKS & FEELS	Agents want to record incidents quickly and accurately. They need clear feedback when a field is required or a value changes.
SEES	The Incident form, Incident list, and fields such as Impact, Urgency, Priority, Assigned to, and State.
SAYS	The user expects the form to explain what needs to be corrected when an input does not meet the configured rules.
DOES	Opens an incident, enters information, changes Impact, saves the record, and may attempt an inline list edit.
PAINS	Missing required values, unclear field relationships, and accidental or invalid changes made from the list.
GAINS	Clear messages, predictable field behaviour, fewer avoidable data-entry errors, and less rework.

2.3 Brainstorming

Three approaches were considered at a conceptual level. Manual checking depends on each user remembering the rules. Server-side validation can enforce data rules during record processing, while client-side UI Policies and Client Scripts can guide the user directly during form interaction.

The documented solution uses a UI Policy and UI Policy Action for Urgency, an onChange Client Script for Impact changes, an onSubmit Client Script for save validation, and an onCellEdit Client Script for the specified list edit. This reflects the configuration components described in the project documentation.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey map

Journey Stage	User Activity / Experience
1. Open Incident	The agent signs in to the ServiceNow instance and opens Incident Management.
2. Enter details	The agent opens an Incident record and enters or reviews the incident information.
3. Change Impact	The agent changes Impact. When Impact is High, the configured logic sets Urgency to 1 - High and the policy makes Urgency read-only.
4. Validate before save	The onSubmit Client Script checks the configured conditions. For a High-impact incident with Assigned to empty, the user sees the required-field message.
5. Edit from list	If the user attempts the specified State change through inline editing, the onCellEdit Client Script displays an alert and prevents that update.
6. Continue workflow	The agent corrects the information if needed and continues using the normal ServiceNow incident workflow.

3.2 Solution Requirement

Functional requirements:

- The “High Impact Control” UI Policy applies the configured field behaviour when its condition is met.
- The UI Policy Action controls the Urgency field and makes it read-only under the configured High-impact condition.
- The onChange Client Script responds to changes in Impact, sets Urgency to 1 - High when Impact is High, and displays a message.
- The onSubmit Client Script validates the configured form conditions before saving.
- When Impact is High and Assigned to is empty, the message “Assigned To is mandatory for High impact incidents.” is displayed and Assigned to is treated as mandatory.
- The onCellEdit Client Script blocks the documented State list edit and displays an explanatory alert.

Environment requirements: a ServiceNow PDI with Incident Management available, a supported web browser, a valid ServiceNow account, and suitable administrator/developer permissions for configuration.

3.3 Data Flow Diagram

Data flow: Agent → ServiceNow Incident Form or Incident List → UI Policy / Client Scripts → field updates and validation → ServiceNow platform → Incident record.

Configuration flow: Administrator configures the UI Policy and Client Scripts within ServiceNow. The platform stores incident data in the incident table and configuration in platform tables such as sys_ui_policy and sys_script_client. No external API or separate project database is specified in the supplied documentation.

3.4 Technology Stack

Component	Technology / Details
Platform	ServiceNow Now Platform / Personal Developer Instance (PDI)
User interface	ServiceNow Incident form and Incident list
Browser	Google Chrome used for the documented manual testing
Client logic	JavaScript and ServiceNow Client Scripts

Client API	g_form for reading/updating fields and displaying messages
Configuration	UI Policy, UI Policy Action, onChange, onSubmit, and onCellEdit Client Scripts
Data storage	ServiceNow incident table and platform configuration tables
Testing method	Manual functional/behaviour testing; no load-testing tool documented

4. PROJECT DESIGN

4.1 Problem Solution Fit

Target users are service-desk agents who create or update Incident records and administrators who maintain the configuration. The problem occurs when a record is entered with missing information, when Impact changes the expected field behaviour, or when a user attempts an inline State edit.

The solution places guidance at the point of interaction. The UI Policy and Client Scripts can make field behaviour clearer, display validation feedback, and stop the specified list update. The solution relies on ServiceNow availability and the roles and permissions configured for the instance.

4.2 Proposed Solution

The proposed solution consists of four configuration areas: (1) the “High Impact Control” UI Policy, (2) a UI Policy Action for Urgency, (3) onChange and onSubmit Client Scripts for form interaction and save validation, and (4) an onCellEdit Client Script for the specified inline list edit.

Documented behaviour: when Impact is High, Urgency is set to 1 - High, Urgency becomes read-only, and Priority shows 1 - Critical. If Impact changes from High to Medium, Urgency becomes editable, Priority changes to 2 - High, and Assigned to is no longer mandatory. When Assigned to is empty for a High-impact incident, the required-field message is displayed.

4.3 Solution Architecture

Layer	Description
Presentation layer	The user accesses the Incident form and Incident list through a web browser.
Client logic layer	The UI Policy, UI Policy Action, and Client Scripts control field behaviour, respond to changes, validate form submission, and check the specified inline edit.
ServiceNow platform layer	ServiceNow manages the user session, platform processing, and incident workflow.
Data layer	Incident records and configuration records are stored within ServiceNow.
Access control	ServiceNow roles and permissions determine who can work with incidents and who can configure policies and scripts.
External services	No external API integration is specified in the supplied project documentation.

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

Sprint / Dates	Planned Activities
Sprint 1 — 1 to 2 October 2026	Set up and verify the ServiceNow environment; create or review the UI Policy on Incident; configure the UI Policy Action for Urgency; perform initial field-behaviour checks.
Sprint 2 — 2 to 3 October 2026	Configure the onChange Client Script and onSubmit Client Script; check Impact changes, save validation, and user-facing messages.
Sprint 3 — 3 October 2026	Configure the onCellEdit Client Script; execute the documented functional scenarios; gather test evidence and complete project documentation.

Team responsibilities are listed on the cover page in the order specified by the user.

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

Testing environment: cloud-hosted ServiceNow Personal Developer Instance, Google Chrome, and a single user. The supplied test record describes manual functional/behaviour testing. No dedicated load-testing tool was used.

Test Scenario	Action / Record	Expected and Reported Result
1. Mandatory field	High impact; Assigned to empty (INC0010006)	Message displayed; Assigned to treated as mandatory.
2. High impact	Set Impact to High	Urgency read-only at 1 - High; Priority 1 - Critical.
3. Medium impact	Change Impact from High to Medium	Urgency editable; Priority 2 - High; Assigned to not mandatory.
4. Inline State edit	Edit State in list (INC0008111)	Alert displayed; State remains unchanged.

Scenario 1 — High impact, Assigned to empty: Record INC0010006. Expected and reported result: the message “Assigned To is mandatory for High impact incidents.” appears and Assigned to is treated as mandatory.

Scenario 2 — Impact set to High: Expected and reported result: Urgency becomes read-only with value 1 - High, and Priority shows 1 - Critical.

Scenario 3 — Impact changed to Medium: Expected and reported result: Urgency becomes editable, Priority changes to 2 - High, and Assigned to is no longer mandatory.

Scenario 4 — State edited from the Incident list: Record INC0008111. Expected and reported result: an alert states that State cannot be updated using list editing and the State value remains unchanged.

Functional test summary: the supplied project testing notes report that all four scenarios behaved as designed during manual checks.

Performance test limitations: response time, throughput, CPU utilisation, memory utilisation, and a formal error rate were not measured. Load, stress, and spike testing were not performed. These performance metrics remain unverified.

7. RESULTS

7.1 Output Screenshots

Insert genuine screenshots captured from the configured ServiceNow instance. The placeholders below identify the evidence to include; they are not screenshots of the running application.

[Screenshot 1: High-impact incident with Assigned to empty and the mandatory-field message.]

[Insert screenshot here]

[Screenshot 2: Impact = High, Urgency = 1 - High, Priority = 1 - Critical, and Urgency read-only.]

[Insert screenshot here]

[Screenshot 3: Impact changed to Medium, Urgency editable, Priority = 2 - High, and Assigned to not mandatory.]

[Insert screenshot here]

[Screenshot 4: Incident list showing the blocked State inline edit and alert.]

[Insert screenshot here]

Result summary: the available evidence supports the documented functional behaviour for the four manual test scenarios. It does not establish performance under concurrent or peak usage.

8. ADVANTAGES & DISADVANTAGES

Advantages

- Provides immediate feedback while users work with Incident records.
- Makes the configured relationship between Impact, Urgency, and Priority clearer.
- Prompts users to provide Assigned to for the documented High-impact condition.
- Blocks the specified State update through inline list editing.
- Uses built-in ServiceNow configuration rather than requiring a separate application stack.

Disadvantages and limitations

- Depends on ServiceNow availability and correct platform configuration.
- Requires appropriate permissions to configure UI Policies and Client Scripts.
- Client-side behaviour should be considered alongside server-side rules and access controls.
- Only manual single-user functional testing is documented; performance under load has not been measured.

9. FUTURE SCOPE

- Conduct controlled performance tests if required, and record actual response times, throughput, error rate, CPU utilisation, and memory utilisation.
- Test additional combinations of Impact, Urgency, Priority, and Assigned to, including empty fields and invalid values.
- Validate the behaviour with different user roles and permissions.
- Review the client-side rules alongside server-side controls to maintain consistent data validation.
- Add final screenshots from the live ServiceNow instance and confirm the team responsibilities before submission.

10. APPENDIX

Source Code (if any)

The project logic is stored as configuration records within ServiceNow rather than as a standalone local application. If required, attach the final UI Policy conditions, UI Policy Action settings, and onChange, onSubmit, and onCellEdit Client Script code. Complete script listings were not included in the supplied reference document, so they are not reproduced here.

Dataset Link

Not applicable to the documented implementation. The project uses ServiceNow Incident records and does not specify a separate dataset.

GitHub & Project Demo Link

GitHub repository: [Insert verified repository URL, if available]

Project demo: [Insert verified demo URL or ServiceNow PDI demonstration details, if available]

No verified GitHub repository or public demo URL was supplied; these fields remain placeholders.